DML for Conditional Differences-in-Differences

Background

We demonstrate the implementation of Differences-in-Differences in an application on the effect of minimum wage changes on teen employment. We use data from Callaway (2022). The data are annual county level data from the United States covering 2001 to 2007. The outcome variable is log county-level teen employment, and the treatment variable is an indicator for whether the county has a minimum wage above the federal minimum wage. Note that this definition of the treatment variable makes the analysis straightforward but ignores the nuances of the exact value of the minimum wage in each county and how far those values are from the federal minimum. The data also include county population and county average annual pay. See Callaway and Sant'Anna (2021) for additional details on the data.

```
library(BMisc)
library(glmnet)
library(randomForest)
library(rpart)
library(xtable)
library(data.table)
set.seed(772023)
options(warn = -1)
```

```
## read data
data <- read.csv("https://raw.githubusercontent.com/CausalAIBook/MetricsMLNotebooks/mai
                 row.names = 1)
data <- data.table(data)
```

Data Preparation

We remove observations that are already treated in the first observed period (2001). We drop all variables that we won't use in our analysis. Next, we create the treatment groups. We focus our analysis exclusively on the set of counties that had wage increases away from the federal minimum wage in 2004. That is, we treat 2003 and earlier as the pre-treatment period.

```
## include never treated and those that have not been treated until 2001
data <- subset(data, (G == 0) | (G > 2001))
data <- data[, -c(
  "countyreal", "state_name", "FIPS", "emp0A01_BS",
  "quarter", "censusdiv", "pop", "annual_avg_pay",
  "state_mw", "fed_mw", "ever_treated"
)]

## treatment group
treat1 <- subset(data, (G == 2004) & (year == 2001))
treat2 <- subset(data, (G == 2004) & (year == 2002))
treat3 <- subset(data, (G == 2004) & (year == 2003))
treat4 <- subset(data, (G == 2004) & (year == 2004))
treat5 <- subset(data, (G == 2004) & (year == 2005))
treat6 <- subset(data, (G == 2004) & (year == 2006))
treat7 <- subset(data, (G == 2004) & (year == 2007))

## control group
cont1 <- subset(data, (G == 0 | G > 2001) & (year == 2001))
cont2 <- subset(data, (G == 0 | G > 2002) & (year == 2002))
cont3 <- subset(data, (G == 0 | G > 2004) & (year == 2003))
cont4 <- subset(data, (G == 0 | G > 2004) & (year == 2004))
cont5 <- subset(data, (G == 0 | G > 2005) & (year == 2005))
cont6 <- subset(data, (G == 0 | G > 2006) & (year == 2006))
cont7 <- subset(data, (G == 0 | G > 2007) & (year == 2007))
```

We assume that the basic assumptions, particularly parallel trends, hold after conditioning on pre-treatment variables: 2001 population, 2001 average pay and 2001 teen employment, as well as the region in which the county is located. Consequently, we

want to extract the control variables for both treatment and control group in 2001. 2003 serves as the pre-treatment period for both counties that do receive the treatment in 2004 and those that do not.

```
treat1 <- treat1[, -c("year", "G", "region", "treated")]
cont1 <- cont1[, -c("year", "G", "region", "treated")]

treatB <- merge(treat3, treat1, by = "id", suffixes = c(".pre", ".0"))
treatB <- treatB[, -c("treated", "lpop.pre", "lavg_pay.pre", "year", "G")]

contB <- merge(cont3, cont1, by = "id", suffixes = c(".pre", ".0"))
contB <- contB[, -c("treated", "lpop.pre", "lavg_pay.pre", "year", "G")]
```

We estimate the ATET in 2004-2007, which corresponds to the effect in the year of treatment as well as in the three years after the treatment. The control observations are the observations that still have the federal minimum wage in each year. (The control group is shrinking in each year as additional units receive treatment).

```
## create Delta Y for treated and controlled
treat4 <- treat4[, -c("lpop", "lavg_pay", "year", "G", "region")]
treat5 <- treat5[, -c("lpop", "lavg_pay", "year", "G", "region")]
treat6 <- treat6[, -c("lpop", "lavg_pay", "year", "G", "region")]
treat7 <- treat7[, -c("lpop", "lavg_pay", "year", "G", "region")]

cont4 <- cont4[, -c("lpop", "lavg_pay", "year", "G", "region")]
cont5 <- cont5[, -c("lpop", "lavg_pay", "year", "G", "region")]
cont6 <- cont6[, -c("lpop", "lavg_pay", "year", "G", "region")]
cont7 <- cont7[, -c("lpop", "lavg_pay", "year", "G", "region")]

tdid04 <- merge(treat4, treatB, by = "id")
dy <- tdid04$lemp - tdid04$lemp.pre
tdid04$dy <- dy
tdid04 <- tdid04[, -c("id", "lemp", "lemp.pre")]

tdid05 <- merge(treat5, treatB, by = "id")
dy <- tdid05$lemp - tdid05$lemp.pre
tdid05$dy <- dy
```

```r
tdid05 <- tdid05[, -c("id", "lemp", "lemp.pre")]

tdid06 <- merge(treat6, treatB, by = "id")
dy <- tdid06$lemp - tdid06$lemp.pre
tdid06$dy <- dy
tdid06 <- tdid06[, -c("id", "lemp", "lemp.pre")]

tdid07 <- merge(treat7, treatB, by = "id")
dy <- tdid07$lemp - tdid07$lemp.pre
tdid07$dy <- dy
tdid07 <- tdid07[, -c("id", "lemp", "lemp.pre")]

cdid04 <- merge(cont4, contB, by = "id")
dy <- cdid04$lemp - cdid04$lemp.pre
cdid04$dy <- dy
cdid04 <- cdid04[, -c("id", "lemp", "lemp.pre")]

cdid05 <- merge(cont5, contB, by = "id")
dy <- cdid05$lemp - cdid05$lemp.pre
cdid05$dy <- dy
cdid05 <- cdid05[, -c("id", "lemp", "lemp.pre")]

cdid06 <- merge(cont6, contB, by = "id")
dy <- cdid06$lemp - cdid06$lemp.pre
cdid06$dy <- dy
cdid06 <- cdid06[, -c("id", "lemp", "lemp.pre")]

cdid07 <- merge(cont7, contB, by = "id")
dy <- cdid07$lemp - cdid07$lemp.pre
cdid07$dy <- dy
cdid07 <- cdid07[, -c("id", "lemp", "lemp.pre")]
```

Estimation of the ATT with DML

We estimate the ATT of the county level minimum wage being larger than the federal minimum with the DML algorithm (presented in Section 16.3) in CML. This requires estimation of the nuisance functions $E[Y \mid D = 0, X]$, $E[D \mid X]$ as well as $P(D = 1)$. For the conditional expectation functions, we will consider different modern ML regression methods, namely: Constant (= no controls); a linear combination of the controls; an expansion of the raw control variables including all third order interactions; Lasso (CV); Ridge (CV); Random Forest; Shallow Tree; Deep Tree; and CV Tree. The methods indicated with CV have their tuning parameter selected by cross-validation.

The following code block implements the DML cross-fitting procedure.

```
## store results
att <- matrix(NA, 4, 10)
se_att <- matrix(NA, 4, 10)
rmse_d <- matrix(NA, 4, 9)
rmse_y <- matrix(NA, 4, 9)
trimmed <- matrix(NA, 4, 9)

for (ii in 1:4) { ## ii refer to the 4 investigated post-treatment periods

  tdata <- get(paste("tdid0", (3 + ii), sep = "")) ## treatment data
  cdata <- get(paste("cdid0", (3 + ii), sep = "")) ## control data
  usedata <- rbind(tdata, cdata)

  #~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~
  ## cross-fit setup
  n <- nrow(usedata)
  Kf <- 5 ## number of folds
  sampleframe <- rep(1:Kf, ceiling(n / Kf))
  cfgroup <- sample(sampleframe, size = n, replace = FALSE) ## cross-fitting groups

  ## initialize variables for cross-fitting predictions
  y_gd0x_fit <- matrix(NA, n, 9)
  dgx_fit <- matrix(NA, n, 9)
  pd_fit <- matrix(NA, n, 1)
```

```r
#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~
## cross-fit loop
for (k in 1:Kf) {
  cat("year: ", ii + 2003, "; fold: ", k, "\n")
  indk <- cfgroup == k

  ktrain <- usedata[!indk, ]
  ktest <- usedata[indk, ]

  ## build Delta Y and treatment D
  ytrain <- as.matrix(usedata[!indk, "dy"])
  ytest <- as.matrix(usedata[indk, "dy"])
  dtrain <- as.matrix(usedata[!indk, "treated"])
  dtest <- as.matrix(usedata[indk, "treated"])

  ## expansion of covariates for lasso/ridge (region specific cubic polynomial)
  Xexpand <- model.matrix(
    ~ region * (polym(lemp.0, lpop.0, lavg_pay.0,
      degree = 3, raw = TRUE
    )),
    data = usedata
  )

  xtrain <- as.matrix(Xexpand[!indk, ])
  xtest <- as.matrix(Xexpand[indk, ])

  #~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~
  ## "estimating" P(D = 1)
  pd_fit[indk, 1] <- mean(ktrain$treated)

  #~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~
  # estimating E[D|X]

  # (1) Constant
  dgx_fit[indk, 1] <- mean(ktrain$treated)
```

```r
    # (2) Baseline controls
    glmXdk <- glm(treated ~ region + lemp.0 + lpop.0 + lavg_pay.0,
      family = "binomial", data = ktrain
    )
    dgx_fit[indk, 2] <- predict(glmXdk, newdata = ktest, type = "response")

    # (3) Region specific linear index
    glmRXdk <- glm(treated ~ region * (lemp.0 + lpop.0 + lavg_pay.0),
      family = "binomial", data = ktrain
    )
    dgx_fit[indk, 3] <- predict(glmRXdk, newdata = ktest, type = "response")

    # (4) Lasso - expansion - default CV tuning
    lassoXdk <- cv.glmnet(xtrain, dtrain, family = "binomial", type.measure = "mse")
    dgx_fit[indk, 4] <- predict(lassoXdk,
      newx = xtest, type = "response",
      s = "lambda.min"
    )

    # (5) Ridge - expansion - default CV tuning
    ridgeXdk <- cv.glmnet(xtrain, dtrain,
      family = "binomial",
      type.measure = "mse", alpha = 0
    )
    dgx_fit[indk, 5] <- predict(ridgeXdk,
      newx = xtest, type = "response",
      s = "lambda.min"
    )

    # (6) Random forest
    rfXdk <- randomForest(as.factor(treated) ~ region + lemp.0 + lpop.0 + lavg_pay.0,
      data = ktrain, mtry = 4, ntree = 1000
    )
    dgx_fit[indk, 6] <- predict(rfXdk, ktest, type = "prob")[, 2]

    # (7) Tree (start big)
```

```r
btXdk <- rpart(treated ~ region + lemp.0 + lpop.0 + lavg_pay.0,
  data = ktrain, method = "anova",
  control = rpart.control(maxdepth = 15, cp = 0, xval = 5, minsplit = 10)
)
# xval is the number of cross-validation splits. E.g. xval = 5 is five fold CV
dgx_fit[indk, 7] <- predict(btXdk, ktest)

# (8) Tree (small tree)
stXdk <- rpart(treated ~ region + lemp.0 + lpop.0 + lavg_pay.0,
  data = ktrain, method = "anova",
  control = rpart.control(maxdepth = 3, cp = 0, xval = 0, minsplit = 10)
)
# xval is the number of cross-validation splits. E.g. xval = 5 is five fold CV
dgx_fit[indk, 8] <- predict(stXdk, ktest)

# (9) Tree (cv)
bestcp <- btXdk$cptable[which.min(btXdk$cptable[, "xerror"]), "CP"]
cvXdk <- prune(btXdk, cp = bestcp)
dgx_fit[indk, 9] <- predict(cvXdk, ktest)

#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~
## estimating E[Y|D=0,X]

## subset to D = 0
ktrain0 <- ktrain[ktrain$treated == 0, ]
ytrain0 <- ytrain[ktrain$treated == 0, ]
xtrain0 <- xtrain[ktrain$treated == 0, ]

# (1) Constant
y_gd0x_fit[indk, 1] <- mean(ktrain0$dy)

# (2) Baseline controls
lmXyk <- lm(dy ~ region + lemp.0 + lpop.0 + lavg_pay.0, data = ktrain0)
y_gd0x_fit[indk, 2] <- predict(lmXyk, newdata = ktest)

# (3) Region specific linear index
```

```r
lmRXyk <- lm(treated ~ region * (lemp.0 + lpop.0 + lavg_pay.0),
  data = ktrain
)
y_gd0x_fit[indk, 3] <- predict(lmRXyk, newdata = ktest)

# (4) Lasso - expansion - default CV tuning
lassoXyk <- cv.glmnet(xtrain0, ytrain0)
y_gd0x_fit[indk, 4] <- predict(lassoXyk, newx = xtest, s = "lambda.min")

# (5) Ridge - expansion - default CV tuning
ridgeXyk <- cv.glmnet(xtrain0, ytrain0, alpha = 0)
y_gd0x_fit[indk, 5] <- predict(ridgeXyk, newx = xtest, s = "lambda.min")

# (6) Random forest
rfXyk <- randomForest(dy ~ region + lemp.0 + lpop.0 + lavg_pay.0,
  data = ktrain0, mtry = 4, ntree = 1000
)
y_gd0x_fit[indk, 6] <- predict(rfXyk, ktest)

# (7) Tree (start big)
btXyk <- rpart(dy ~ region + lemp.0 + lpop.0 + lavg_pay.0,
  data = ktrain0, method = "anova",
  control = rpart.control(maxdepth = 15, cp = 0, xval = 5, minsplit = 10)
)
y_gd0x_fit[indk, 7] <- predict(btXyk, ktest)

# (8) Tree (small tree)
stXyk <- rpart(dy ~ region + lemp.0 + lpop.0 + lavg_pay.0,
  data = ktrain, method = "anova",
  control = rpart.control(maxdepth = 3, cp = 0, xval = 0, minsplit = 10)
)
y_gd0x_fit[indk, 8] <- predict(stXyk, ktest)

# (9) Tree (cv)
bestcp <- btXyk$cptable[which.min(btXyk$cptable[, "xerror"]), "CP"]
cvXyk <- prune(btXyk, cp = bestcp)
```

```
    y_gd0x_fit[indk, 9] <- predict(cvXyk, ktest)
  }

  rmse_d[ii, ] <- sqrt(colMeans((usedata$treated - dgx_fit)^2))
  rmse_y[ii, ] <- sqrt(colMeans((usedata$dy[usedata$treated == 0] -
                                 y_gd0x_fit[usedata$treated == 0, ])^2))

  ## trim propensity scores of 1 to .95
  for (r in 1:9) {
    trimmed[ii, r] <- sum(dgx_fit[, r] > .95)
    dgx_fit[dgx_fit[, r] > .95, r] <- .95
  }

  ## save results
  att_num <- c(
    colMeans(((usedata$treated - dgx_fit) / ((pd_fit %*% matrix(1, 1, 9)) * (1 - dgx_fi
               (usedata$dy - y_gd0x_fit)),
    mean(((usedata$treated - dgx_fit[, which.min(rmse_d[ii, ])])
          / (pd_fit * (1 - dgx_fit[, which.min(rmse_d[ii, ])]))) *
           (usedata$dy - y_gd0x_fit[, which.min(rmse_y[ii, ])]))
  )
  att_den <- mean(usedata$treated / pd_fit)

  ## calculate ATT
  att[ii, ] <- att_num / att_den

  ## calculate SE
  phihat <- cbind(
    ((usedata$treated - dgx_fit) / ((pd_fit %*% matrix(1, 1, 9)) * (1 - dgx_fit))) *
      (usedata$dy - y_gd0x_fit),
    ((usedata$treated - dgx_fit[, which.min(rmse_d[ii, ])])
     / (pd_fit * (1 - dgx_fit[, which.min(rmse_d[ii, ])]))) *
      (usedata$dy - y_gd0x_fit[, which.min(rmse_y[ii, ])])
  ) / att_den
  se_att[ii, ] <- sqrt(colMeans((phihat^2)) / n)
}
```

```
year:  2004 ; fold:  1
year:  2004 ; fold:  2
year:  2004 ; fold:  3
year:  2004 ; fold:  4
year:  2004 ; fold:  5
year:  2005 ; fold:  1
year:  2005 ; fold:  2
year:  2005 ; fold:  3
year:  2005 ; fold:  4
year:  2005 ; fold:  5
year:  2006 ; fold:  1
year:  2006 ; fold:  2
year:  2006 ; fold:  3
year:  2006 ; fold:  4
year:  2006 ; fold:  5
year:  2007 ; fold:  1
year:  2007 ; fold:  2
year:  2007 ; fold:  3
year:  2007 ; fold:  4
year:  2007 ; fold:  5
```

We start by reporting the RMSE obtained during cross-fitting for each learner in each period.

```r
## report RMSE for delta Y
table1y <- matrix(0, 9, 4)
table1y <- t(rmse_y)
colnames(table1y) <- c("2004", "2005", "2006", "2007")
rownames(table1y) <- c(
  "No Controls", "Basic", "Expansion",
  "Lasso (CV)", "Ridge (CV)",
  "Random Forest", "Deep Tree",
  "Shallow Tree", "Tree (CV)"
)
print(table1y)
```

	2004	2005	2006	2007

```
No Controls   0.1633223 0.1881749 0.2233685 0.2302511
Basic         0.1634008 0.1854070 0.2182060 0.2215877
Expansion     0.1887055 0.2123235 0.2446543 0.2699818
Lasso (CV)    0.1630260 0.1855099 0.2185013 0.2213843
Ridge (CV)    0.1630414 0.1853066 0.2181571 0.2213704
Random Forest 0.1715711 0.1962291 0.2306600 0.2419245
Deep Tree     0.1922407 0.2258910 0.2568412 0.2733272
Shallow Tree  0.1677878 0.1924367 0.2240679 0.2247410
Tree (CV)     0.1633223 0.1880679 0.2175507 0.2228741
```

```r
## report RMSE for D
table1d <- matrix(0, 9, 4)
table1d <- t(rmse_d)
colnames(table1d) <- c("2004", "2005", "2006", "2007")
rownames(table1d) <- c(
  "No Controls", "Basic", "Expansion",
  "Lasso (CV)", "Ridge (CV)",
  "Random Forest", "Deep Tree",
  "Shallow Tree", "Tree (CV)"
)
print(table1d)
```

```
                  2004      2005      2006      2007
No Controls   0.1982598 0.2007186 0.2111847 0.2504211
Basic         0.1986435 0.2014720 0.2112553 0.2198657
Expansion     0.1988052 0.2015538 0.2112478 0.2198657
Lasso (CV)    0.1968467 0.1987782 0.2083413 0.2189074
Ridge (CV)    0.1971383 0.1992067 0.2085501 0.2191746
Random Forest 0.2005306 0.2055452 0.2080690 0.2304464
Deep Tree     0.2206856 0.2337925 0.2347630 0.2639295
Shallow Tree  0.1921123 0.1960193 0.2021778 0.2285372
Tree (CV)     0.1933313 0.1957681 0.2050068 0.2321018
```

Here we see that the Deep Tree systematically performs worse in terms of cross-fit predictions than the other learners for both tasks and that Expansion performs similarly poorly for the outcome prediction. It also appears there is some signal in the regressors,

especially for the propensity score, as all methods outside of Deep Tree and Expansion produce smaller RMSEs than the No Controls baseline. The other methods all produce similar RMSEs, with a small edge going to Ridge and Lasso. While it would be hard to reliably conclude which of the relatively good performing methods is statistically best here, one could exclude Expansion and Deep Tree from further consideration on the basis of out-of-sample performance suggesting they are doing a poor job approximating the nuisance functions. Best (or a different ensemble) provides a good baseline that is principled in the sense that one could pre-commit to using the best learners without having first looked at the subsequent estimation results.

We report estimates of the ATET in each period in the following table.

```
## report ATT
table2 <- matrix(0, 20, 4)
table2[seq(1, 20, 2), ] <- t(att)
table2[seq(2, 20, 2), ] <- t(se_att)
colnames(table2) <- c("2004", "2005", "2006", "2007")
rownames(table2) <- c(
  "No Controls", "s.e.", "Basic", "s.e.",
  "Expansion", "s.e.", "Lasso (CV)", "s.e.",
  "Ridge (CV)", "s.e.", "Random Forest", "s.e.",
  "Deep Tree", "s.e.", "Shallow Tree", "s.e.",
  "Tree (CV)", "s.e.", "Best", "s.e."
)
table2
```

```
                    2004        2005        2006        2007
No Controls   -0.03929934 -0.07537546 -0.11588624 -0.13131724
s.e.           0.01868421  0.02124910  0.02271154  0.02621935
Basic         -0.03663512 -0.06503700 -0.09038017 -0.04380777
s.e.           0.01842984  0.02056043  0.02105421  0.03077225
Expansion     -0.02243204 -0.04069853 -0.06291078  0.21562540
s.e.           0.02499826  0.03017896  0.03242890  0.19216424
Lasso (CV)    -0.03427772 -0.06140426 -0.08207244 -0.05833393
s.e.           0.01844538  0.02027314  0.02073762  0.03159693
Ridge (CV)    -0.03477344 -0.06144218 -0.08335489 -0.06255241
s.e.           0.01845780  0.02023796  0.02074374  0.02536365
```

```
Random Forest  0.01541716 -0.05643210 -0.06385188 -0.08187092
s.e.           0.02981586  0.02449611  0.02559016  0.04624743
Deep Tree      0.07732570  0.05353250  0.11201144  0.09633844
s.e.           0.07899597  0.13078707  0.16676550  0.11440006
Shallow Tree  -0.02788830 -0.04929880 -0.05436142 -0.07383930
s.e.           0.01893650  0.02095667  0.02093805  0.02676119
Tree (CV)     -0.02544778 -0.04317841 -0.03066357 -0.07803380
s.e.           0.01880279  0.02118395  0.02908569  0.02663734
Best          -0.02782004 -0.05471893 -0.05214769 -0.05317516
s.e.           0.01866244  0.02053891  0.02104099  0.03134227
```

Here, we see that the majority of methods provide point estimates that suggest the effect of the minimum wage increase leads to decreases in youth employment with small effects in the initial period that become larger in the years following the treatment. This pattern seems economically plausible as it may take time for firms to adjust employment and other input choices in response to a minimum wage change. The methods that produce estiamtes that are not consistent with this pattern are Deep Tree and Expansion which are both suspect as they systematically underperform in terms of having poor cross-fit prediction performance. In terms of point estimates, the other pattern that emerges is that all estimates that use the covariates produce ATT estimates that are systematically smaller in magnitude than the No Controls baseline, suggesting that failing to include the controls may lead to overstatement of treatment effects in this example.

Turning to inference, we would reject the hypothesis of no minimum wage effect two or more years after the change at the 5% level, even after multiple testing correction, if we were to focus on the row "Best" (or many of the other individual rows). Focusing on "Best" is a reasonable ex ante strategy that could be committed to prior to conducting any analysis. It is, of course, reassuring that this broad conclusion is also obtained using many of the individual learners suggesting some robustness to the exact choice of learner made.

Assess pre-trends

Because we have data for the period 2001-2007, we can perform a so-called pre-trends test to provide some evidence about the plausibility of the conditional parallel trends assumption. Specifically, we can continue to use 2003 as the reference period but now

consider 2002 to be the treatment period. Sensible economic mechanisms underlying the assumption would then typically suggest that the ATET in 2002 - before the 2004 minimum wage change we are considering - should be zero. Finding evidence that the ATET in 2002 is non-zero then calls into question the validity of the assumption.

We change the treatment status of those observations, which received treatment in 2004 in the 2002 data and create a placebo treatment as well as control group.

```
treat2 <- treat2[, -c("lpop", "lavg_pay", "year", "G", "region")]
treat2$treated <- 1 ## code these observations as treated

tdid02 <- merge(treat2, treatB, by = "id")
dy <- tdid02$lemp - tdid02$lemp.pre
tdid02$dy <- dy
tdid02 <- tdid02[, -c("id", "lemp", "lemp.pre")]

cont2 <- cont2[, -c("lpop", "lavg_pay", "year", "G", "region")]

cdid02 <- merge(cont2, contB, by = "id")
dy <- cdid02$lemp - cdid02$lemp.pre
cdid02$dy <- dy
cdid02 <- cdid02[, -c("id", "lemp", "lemp.pre")]
```

We repeat the exercise for obtaining our ATE estimates and standard error for 2004-2007. Particularly, we also use all the learners as mentioned above.

```
## store results
att_p <- matrix(NA, 1, 10)
se_att_p <- matrix(NA, 1, 10)
rmse_d_p <- matrix(NA, 1, 9)
rmse_y_p <- matrix(NA, 1, 9)
trimmed_p <- matrix(NA, 1, 9)
for (ii in 1) {
  tdata <- get(paste("tdid0", (3 - ii), sep = "")) ## treatment data
  cdata <- get(paste("cdid0", (3 - ii), sep = "")) ## control data
  usedata <- rbind(tdata, cdata)
```

```r
#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~
## cross-fit setup
n <- nrow(usedata)
Kf <- 5 # Number of folds
sampleframe <- rep(1:Kf, ceiling(n / Kf))
cfgroup <- sample(sampleframe, size = n, replace = FALSE) ## cross-fitting groups

## initialize variables for CV predictions
y_gd0x_fit <- matrix(NA, n, 9)
dgx_fit <- matrix(NA, n, 9)
pd_fit <- matrix(NA, n, 1)

#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~
## cross-fit loop
for (k in 1:Kf) {
  cat("year: ", ii + 2001, "; fold: ", k, "\n")
  indk <- cfgroup == k

  ktrain <- usedata[!indk, ]
  ktest <- usedata[indk, ]

  ## build some matrices for later
  ytrain <- as.matrix(usedata[!indk, "dy"])
  ytest <- as.matrix(usedata[indk, "dy"])
  dtrain <- as.matrix(usedata[!indk, "treated"])
  dtest <- as.matrix(usedata[indk, "treated"])

  ## expansion for lasso/ridge (region specific cubic polynomial)
  Xexpand <- model.matrix(
    ~ region * (polym(lemp.0, lpop.0, lavg_pay.0,
      degree = 3, raw = TRUE
    )),
    data = usedata
  )

  xtrain <- as.matrix(Xexpand[!indk, ])
```

```r
xtest <- as.matrix(Xexpand[indk, ])

#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~
## estimating P(D = 1)
pd_fit[indk, 1] <- mean(ktrain$treated)

#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~
## estimating E[D|X]

# (1) Constant
dgx_fit[indk, 1] <- mean(ktrain$treated)

# (2) Baseline controls
glmXdk <- glm(treated ~ region + lemp.0 + lpop.0 + lavg_pay.0,
  family = "binomial", data = ktrain
)
dgx_fit[indk, 2] <- predict(glmXdk, newdata = ktest, type = "response")

# (3) Region specific linear index
glmRXdk <- glm(treated ~ region * (lemp.0 + lpop.0 + lavg_pay.0),
  family = "binomial", data = ktrain
)
dgx_fit[indk, 3] <- predict(glmRXdk, newdata = ktest, type = "response")

# (4) Lasso - expansion - default CV tuning
lassoXdk <- cv.glmnet(xtrain, dtrain, family = "binomial", type.measure = "mse")
dgx_fit[indk, 4] <- predict(lassoXdk,
  newx = xtest, type = "response",
  s = "lambda.min"
)

# (5) Ridge - expansion - default CV tuning
ridgeXdk <- cv.glmnet(xtrain, dtrain,
  family = "binomial",
  type.measure = "mse", alpha = 0
)
```

```r
dgx_fit[indk, 5] <- predict(ridgeXdk,
  newx = xtest, type = "response",
  s = "lambda.min"
)

# (6) Random forest
rfXdk <- randomForest(as.factor(treated) ~ region + lemp.0 + lpop.0 + lavg_pay.0,
  data = ktrain, mtry = 4, ntree = 1000
)
dgx_fit[indk, 6] <- predict(rfXdk, ktest, type = "prob")[, 2]

# (7) Tree (start big)
btXdk <- rpart(treated ~ region + lemp.0 + lpop.0 + lavg_pay.0,
  data = ktrain, method = "anova",
  control = rpart.control(maxdepth = 15, cp = 0, xval = 5, minsplit = 10)
)
# xval is the number of cross-validation splits. E.g. xval = 5 is five fold CV
dgx_fit[indk, 7] <- predict(btXdk, ktest)

# (8) Tree (small tree)
stXdk <- rpart(treated ~ region + lemp.0 + lpop.0 + lavg_pay.0,
  data = ktrain, method = "anova",
  control = rpart.control(maxdepth = 3, cp = 0, xval = 0, minsplit = 10)
)
# xval is the number of cross-validation splits. E.g. xval = 5 is five fold CV
dgx_fit[indk, 8] <- predict(stXdk, ktest)

# (9) Tree (cv)
bestcp <- btXdk$cptable[which.min(btXdk$cptable[, "xerror"]), "CP"]
cvXdk <- prune(btXdk, cp = bestcp)
dgx_fit[indk, 9] <- predict(cvXdk, ktest)

#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~#~
## estimating E[Y|D=0,X]

# subset to D = 0
```

```r
ktrain0 <- ktrain[ktrain$treated == 0, ]

ytrain0 <- ytrain[ktrain$treated == 0, ]
xtrain0 <- xtrain[ktrain$treated == 0, ]

# (1) Constant
y_gd0x_fit[indk, 1] <- mean(ktrain0$dy)

# (2) Baseline controls
lmXyk <- lm(dy ~ region + lemp.0 + lpop.0 + lavg_pay.0, data = ktrain0)
y_gd0x_fit[indk, 2] <- predict(lmXyk, newdata = ktest)

# (3) Region specific linear index
lmRXyk <- lm(treated ~ region * (lemp.0 + lpop.0 + lavg_pay.0),
  data = ktrain
)
y_gd0x_fit[indk, 3] <- predict(lmRXyk, newdata = ktest)

# (4) Lasso - expansion - default CV tuning
lassoXyk <- cv.glmnet(xtrain0, ytrain0)
y_gd0x_fit[indk, 4] <- predict(lassoXyk, newx = xtest, s = "lambda.min")

# (5) Ridge - expansion - default CV tuning
ridgeXyk <- cv.glmnet(xtrain0, ytrain0, alpha = 0)
y_gd0x_fit[indk, 5] <- predict(ridgeXyk, newx = xtest, s = "lambda.min")

# (6) Random forest
rfXyk <- randomForest(dy ~ region + lemp.0 + lpop.0 + lavg_pay.0,
  data = ktrain0, mtry = 4, ntree = 1000
)
y_gd0x_fit[indk, 6] <- predict(rfXyk, ktest)

# (7) Tree (start big)
btXyk <- rpart(dy ~ region + lemp.0 + lpop.0 + lavg_pay.0,
  data = ktrain0, method = "anova",
  control = rpart.control(maxdepth = 15, cp = 0, xval = 5, minsplit = 10)
```

```
  )
  y_gd0x_fit[indk, 7] <- predict(btXyk, ktest)

  # (8) Tree (small tree)
  stXyk <- rpart(dy ~ region + lemp.0 + lpop.0 + lavg_pay.0,
    data = ktrain, method = "anova",
    control = rpart.control(maxdepth = 3, cp = 0, xval = 0, minsplit = 10)
  )
  y_gd0x_fit[indk, 8] <- predict(stXyk, ktest)

  # (9) Tree (cv)
  bestcp <- btXyk$cptable[which.min(btXyk$cptable[, "xerror"]), "CP"]
  cvXyk <- prune(btXyk, cp = bestcp)
  y_gd0x_fit[indk, 9] <- predict(cvXyk, ktest)
}

rmse_d_p[ii, ] <- sqrt(colMeans((usedata$treated - dgx_fit)^2))
rmse_y_p[ii, ] <- sqrt(colMeans((usedata$dy[usedata$treated == 0] -
                                   y_gd0x_fit[usedata$treated == 0, ])^2))

## trim propensity scores of 1 to .95
for (r in 1:9) {
  trimmed_p[ii, r] <- sum(dgx_fit[, r] > .95)
  dgx_fit[dgx_fit[, r] > .95, r] <- .95
}

att_num <- c(
  colMeans(((usedata$treated - dgx_fit) / ((pd_fit %*% matrix(1, 1, 9)) * (1 - dgx_fi
             (usedata$dy - y_gd0x_fit)),
  mean(((usedata$treated - dgx_fit[, which.min(rmse_d[ii, ])])
        / (pd_fit * (1 - dgx_fit[, which.min(rmse_d[ii, ])]))) *
         (usedata$dy - y_gd0x_fit[, which.min(rmse_y[ii, ])]))
)
att_den <- mean(usedata$treated / pd_fit)

att_p[ii, ] <- att_num / att_den
```

```
  phihat <- cbind(
    ((usedata$treated - dgx_fit) / ((pd_fit %*% matrix(1, 1, 9)) * (1 - dgx_fit))) *
      (usedata$dy - y_gd0x_fit),
    ((usedata$treated - dgx_fit[, which.min(rmse_d[ii, ])])
     / (pd_fit * (1 - dgx_fit[, which.min(rmse_d[ii, ])]))) *
      (usedata$dy - y_gd0x_fit[, which.min(rmse_y[ii, ])])
  ) / att_den
  se_att_p[ii, ] <- sqrt(colMeans((phihat^2)) / n)
}
```

```
year:  2002 ; fold:  1
year:  2002 ; fold:  2
year:  2002 ; fold:  3
year:  2002 ; fold:  4
year:  2002 ; fold:  5
```

```
tableP <- matrix(0, 4, 10)
tableP[1, ] <- c(rmse_y_p, min(rmse_y_p))
tableP[2, ] <- c(rmse_d_p, min(rmse_d_p))
tableP[3, ] <- att_p
tableP[4, ] <- se_att_p
rownames(tableP) <- c("RMSE Y", "RMSE D", "ATET", "s.e.")
colnames(tableP) <- c(
  "No Controls", "Basic", "Expansion",
  "Lasso (CV)", "Ridge (CV)",
  "Random Forest", "Deep Tree",
  "Shallow Tree", "Tree (CV)", "Best"
)
tableP <- t(tableP)
tableP
```

```
                RMSE Y    RMSE D          ATET       s.e.
No Controls  0.1552841 0.1982179 -0.0053513289 0.01330981
Basic        0.1551030 0.1985121 -0.0050875034 0.01317136
Expansion    0.1591141 0.1983789  0.0022641259 0.01450300
```

```
Lasso (CV)    0.1559935 0.1968064 -0.0047872018 0.01321668
Ridge (CV)    0.1556025 0.1971063 -0.0058636900 0.01319686
Random Forest 0.1638637 0.2045336  0.0066380873 0.01885076
Deep Tree     0.1844021 0.2292839 -0.0112021959 0.09241534
Shallow Tree  0.1612955 0.1916421 -0.0024544461 0.01395667
Tree (CV)     0.1570763 0.1968843 -0.0025779189 0.01391759
Best          0.1551030 0.1916421  0.0005740152 0.01379214
```

Here we see broad agreement across all methods in the sense of returning point estimates that are small in magnitude and small relative to standard errors. In no case would we reject the hypothesis that the pre-event effect in 2002 is different from zero at usual levels of significance. We note that failing to reject the hypothesis of no pre-event effects certainly does not imply that the conditional DiD assumption is in fact satisfied. For example, confidence intervals include values that would be consistent with relatively large pre-event effects. However, it is reassuring to see that there is not strong evidence of a violation of the underlying identifying assumption.